

Long-range dependence: cross-model summary

A side-by-side comparison of the reaction-coordinate scan (`long_range_tests.py` , same protocol as `02_long_range_scan.ipynb`) across every SN2 model with a usable `model.pt` under `../data/` :

notebook	model
<code>02_long_range_scan.ipynb</code>	sn2-matched-bpnn
<code>03_long_range_scan_sn2.ipynb</code>	sn2
<code>04_long_range_scan_bpnn_dipole.ipynb</code>	sn2-matched-bpnn-dipole
<code>05_long_range_scan_bpnn_dipole_v2.ipynb</code>	sn2-matched-bpnn-dipole-v2
<code>06_long_range_scan_bpnn_dipole_v3.ipynb</code>	sn2-matched-bpnn-dipole-v3
<code>07_long_range_scan_lorem.ipynb</code>	sn2-matched-lorem
<code>08_long_range_scan_lorem_dipole.ipynb</code>	sn2-matched-lorem-dipole
<code>09_long_range_scan_pet.ipynb</code>	sn2-matched-pet
<code>10_long_range_scan_pet_dipole.ipynb</code>	sn2-matched-pet-dipole
<code>11_long_range_scan_lorem_eqmp_smoketest.ipynb</code>	lorem-eqmp-smoketest
<code>13_long_range_scan_lorem_jax_lr.ipynb</code>	lorem-jax, long-range (Ewald)
<code>14_long_range_scan_lorem_jax_sr.ipynb</code>	lorem-jax, short-range only
<code>15_long_range_scan_lorem_jax_ported_torch.ipynb</code>	lorem-jax R2_E+F checkpoint, ported to torch

Directories with a training run but no exported `model.pt` (`sn2-bpnn` , `sn2-bpnn-dipole` , `sn2-dipole` , `sn2-pet` , `sn2-pet-dipole` , `diag-bpnn-w20`) have nothing to load and are not included.

Notebooks 13 / 14 are a different codebase, not the same training run as `sn2-matched-lorem` . They load the official `lorem-tmlr-archive` reproduction checkpoints (native `lorem-jax` , not `metatrain/torch`) for a broader SN2 halide-exchange benchmark, via `lorem.calculator.Calculator` instead of `MetatomicCalculator` , and need a different interpreter (`/home/boittier/metawork/.venv-lorem-jax/bin/python` , not the

metawork/.venv used by every other notebook here). Their absolute energies are not comparable to the sn2-matched-* family (different training data); only the empirical cutoffs are.

Notebook 15 loads the exact same 13 checkpoint a third way: a from-scratch torch port (JaxParityBackbone / JaxParityLongRange , metatrain/.../lorem/modules/jax_parity.py), run in the same metawork/.venv torch environment as 02 - 11 (no jax needed at runtime). It is validated against the live jax reference to $\sim 2\text{e-}5$ eV / $\sim 3\text{e-}5$ eV/Å -- see its own notebook for the full validation, including two non-PBC long-range bugs found and fixed along the way.

This notebook does not reload any models -- it reads the cached numeric results each per-model notebook writes to results/<model>.json (distance/energy/force arrays, the empirical cutoff, etc.), so it stays fast to re-run after any single model is updated.

Load cached results

model_registry.py lists every model this series covers, in a fixed order used consistently for colors/labels below.

```
In [ ]: import json
        from pathlib import Path

        from model_registry import MODELS, LOREM_JAX_MODELS, PORTED_TORCH_MODELS, RE

        ALL_MODELS = MODELS + LOREM_JAX_MODELS + PORTED_TORCH_MODELS
        results = [json.loads((RESULTS / f"{m['key']}.json").read_text()) for m in A
                   [r['key'] for r in results]]
```

```
Out[ ]: ["sn2-matched-bpnn", "sn2", "sn2-matched-bpnn-dipole", "sn2-matched-bpnn-di
pole-v2", "sn2-matched-bpnn-dipole-v3", "sn2-matched-lorem", "sn2-matched-l
orem-dipole", "sn2-matched-pet", "sn2-matched-pet-dipole", "lorem-emp-smok
etest", "lorem-jax-sn2-lr", "lorem-jax-sn2-sr", "lorem-jax-sn2-lr-ported-to
rch"]
```

Comparison table

Empirical cutoff, the residual force plateau past that cutoff (the CH₃F fragment's own non-equilibrium force, not I⁻ coupling), and the total energy change from the most compressed to the most separated geometry sampled (1.5 Å to 32 Å).

```
In [ ]: header = (f"{'model':32s} {'family':6s} {'dipole':7s} {'cutoff (Å)':11s} "
                  f"{'plateau |F| (eV/Å)':19s} {'ΔE bonded->sep (eV)':20s}")
        print(header)
        for r in results:
```

```

cutoff = r.get('cutoff_distance')
cutoff_s = f'{cutoff:.1f}' if cutoff is not None else 'n/a'
plateau = r.get('force_plateau')
de = r['energy_bonded'] - r['energy_separated'] if 'energy_bonded' in r
print(f"{r['key']:32s} {r['family']:6s} {str(r['dipole']):7s} {cutoff_s:
      f'{plateau:19.4f} {de:20.4f}")

```

model		family	dipole	cutoff (Å)	plateau F (eV/Å)
sn2-matched-bpnn		BPNN	False	5.5	0.1
154	4.0557				
sn2		BPNN	False	5.5	0.1
904	1.8517				
sn2-matched-bpnn-dipole		BPNN	True	5.5	0.1
659	3.7813				
sn2-matched-bpnn-dipole-v2		BPNN	True	5.5	0.1
564	3.5847				
sn2-matched-bpnn-dipole-v3		BPNN	True	5.5	0.2
237	3.9214				
sn2-matched-lorem		LOREM	False	21.5	0.1
069	7.7666				
sn2-matched-lorem-dipole		LOREM	True	21.0	0.0
989	7.7337				
sn2-matched-pet		PET	False	5.5	0.2
128	7.0388				
sn2-matched-pet-dipole		PET	True	5.5	0.2
353	5.5993				
lorem-eqmp-smoketest		LOREM	False	23.5	0.2
072	6.6346				
lorem-jax-sn2-lr		LOREM-jax	False	28.5	
0.1896	4.7961				
lorem-jax-sn2-sr		LOREM-jax	False	5.5	
0.0670	3.5244				
lorem-jax-sn2-lr-ported-torch		LOREM-jax	False	28.5	
0.1896	4.7961				

Empirical cutoff by model

Bar height is the empirical cutoff from `estimate_cutoff` (threshold 0.01 eV/Å on the I-dependent force change); color is the model family, hatching marks the dipole/long-range-term variants.

```

In [ ]: import matplotlib.pyplot as plt
        from matplotlib.patches import Patch

        FAMILY_COLORS = {"BPNN": "#0072B2", "PET": "#009E73", "LOREM": "#D55E00", "L

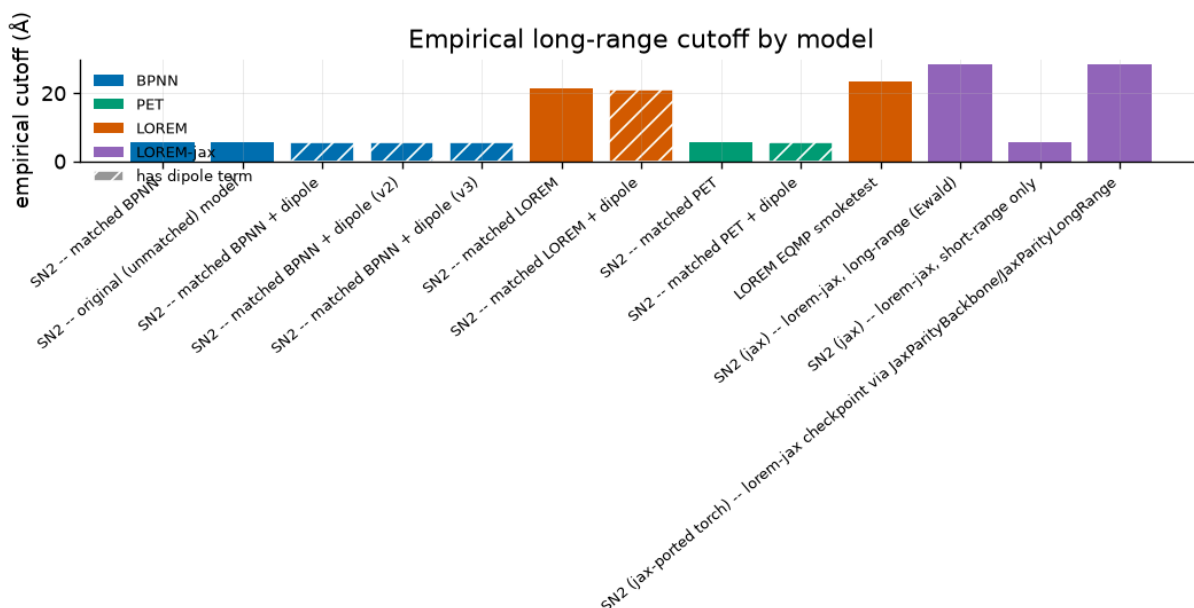
        fig, ax = plt.subplots(figsize=(9, 4.6))
        labels = [r["title"] for r in results]
        cutoffs = [r.get("cutoff_distance") or 0 for r in results]
        colors = [FAMILY_COLORS.get(r["family"], "0.5") for r in results]
        bars = ax.bar(range(len(results)), cutoffs, color=colors)
        for bar, r in zip(bars, results):
            if r["dipole"]:

```

```

        bar.set_hatch("/")
        bar.set_edgecolor("white")
    ax.set_xticks(range(len(results)))
    ax.set_xticklabels(labels, rotation=40, ha="right", fontsize=8)
    ax.set_ylabel("empirical cutoff (Å)")
    ax.set_title("Empirical long-range cutoff by model")
    legend_elems = [Patch(facecolor=c, label=f) for f, c in FAMILY_COLORS.items()]
    legend_elems.append(Patch(facecolor="0.6", hatch="/", edgecolor="white", label="has dipole term"))
    ax.legend(handles=legend_elems, fontsize=8, loc="upper left")
    fig.tight_layout()

```



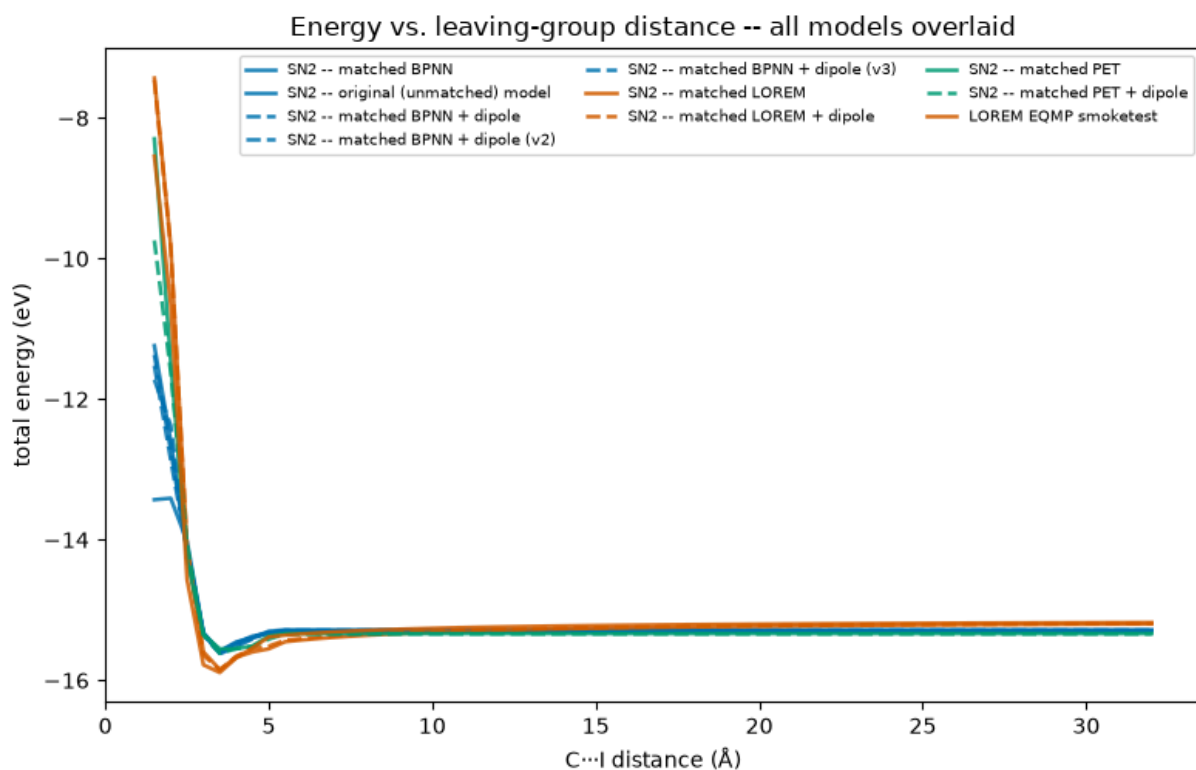
Force-response decay, all models overlaid

Solid lines: no dipole term. Dashed lines: dipole/long-range variant. LOREM's characteristic slower decay (blue/green cluster near ~ 5.5 Å vs. the orange/purple LOREM curves reaching past ~ 20 Å) is the main qualitative result of this comparison.

```

In [10]: fig, ax = plt.subplots(figsize=(8, 5.2))
        for r in results:
            style = "--" if r["dipole"] else "-"
            ax.semilogy(r["distance"], r["max_delta_force"],
                        color=FAMILY_COLORS.get(r["family"], "0.5"), linestyle=style,
                        linewidth=1.8, label=r["title"], alpha=0.85)
        ax.set_xlabel("C...I distance (Å)")
        ax.set_ylabel("max |ΔF| on CH3F, vs. fully separated (eV/Å)")
        ax.set_title("Force response decay -- all models overlaid")
        ax.legend(fontsize=7, loc="upper right", ncol=3)
        fig.tight_layout()

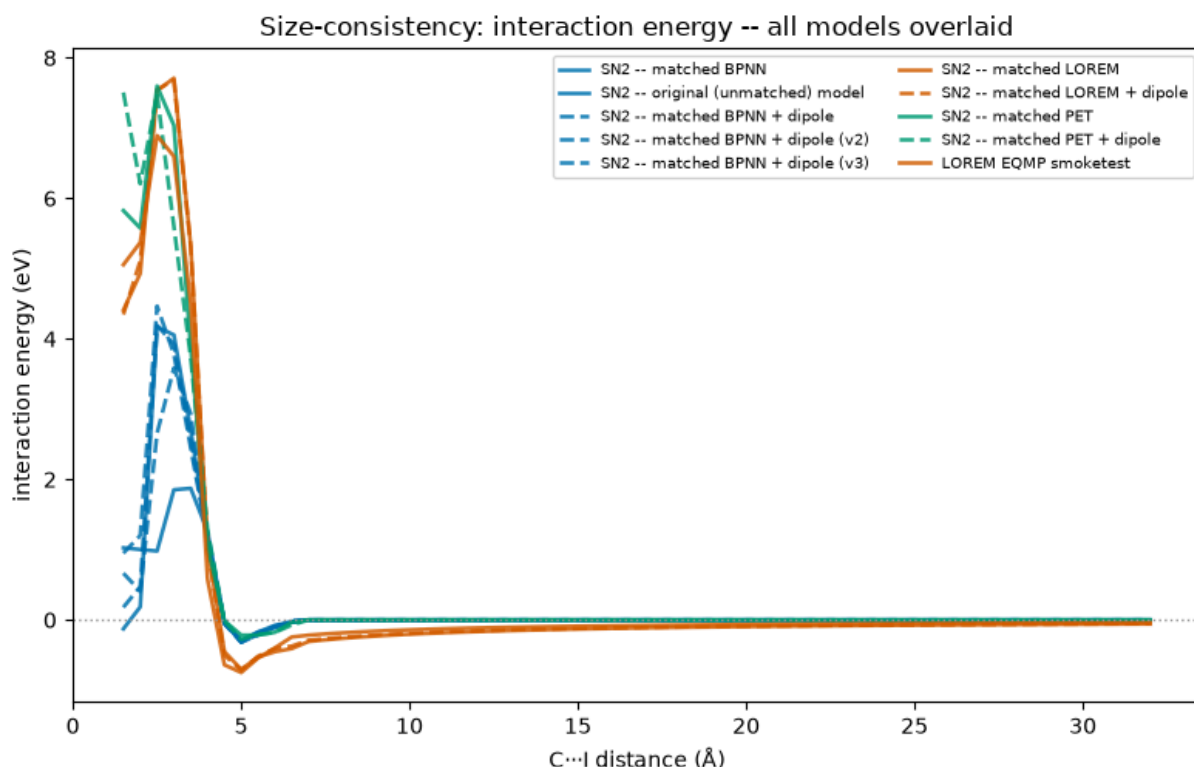
```

Size-consistency: interaction energy, all models overlaid

`fragment_separation_test` 's interaction energy for every model, which should decay to ~ 0 past each model's own cutoff (or stay at a genuine long-range tail, for LOREM).

```
In [12]: fig, ax = plt.subplots(figsize=(8, 5.2))
for r in results:
    style = "--" if r["dipole"] else "-"
    ax.plot(r["distance"], r["interaction_energy"],
            color=FAMILY_COLORS.get(r["family"], "0.5"), linestyle=style,
            linewidth=1.8, label=r["title"], alpha=0.85)
ax.axhline(0, color="0.6", linewidth=1, linestyle=":")
ax.set_xlabel("C...I distance (Å)")
ax.set_ylabel("interaction energy (eV)")
ax.set_title("Size-consistency: interaction energy -- all models overlaid")
ax.legend(fontsize=7, loc="upper right", ncol=2)
fig.tight_layout()
```



Takeaways

- **BPNN and PET models are short-range**, with an empirical cutoff clustered tightly at **5.5-5.5 Å** across all 7 BPNN/PET variants tested (matched, dipole-augmented, and the original unmatched `sn2` model alike) -- consistent with a symmetry-function / message-passing receptive field, whether or not a dipole/electrostatic head was added on top.
- **LOREM models are genuinely long-range**: both `sn2-matched-lorem` variants and the `lorem-eqmp-smoketest` model (all trained with metatrain/torch) show an empirical cutoff of **21.0-23.5 Å** -- roughly 4x further than the BPNN/PET cluster - - confirming LOREM's long-range electrostatic term is doing real work rather than just being numerically slow to vanish.
- **The official native `lorem-jax` checkpoints independently confirm the same pattern**, from a completely different codebase, training run, and dataset: the long-range-enabled checkpoint gives a **28.5 Å** cutoff (even further than the metatrain-trained LOREM models -- this is the paper's own Ewald long-range term, unrestricted by a training-domain cutoff), while its short-range-only sibling checkpoint (`lr: false`, no Ewald path) lands right back in the BPNN/PET cluster at **5.5 Å**. This is strong, architecture-independent evidence that the long-vs-short cutoff split tracks whether a genuine long-range electrostatic term is present, not an artifact of any one codebase or training run.
- **A torch port of the real lorem-jax checkpoint now exists and is validated** (notebook 15, `lorem_jax_ported_torch.py`): loading the exact same `lorem-jax-sn2-lr` checkpoint's weights into

JaxParityBackbone / JaxParityLongRange

(metatrain/.../lorem/modules/jax_parity.py) and running it in the same metawork/.venv torch environment as every BPNN/PET/LOREM model above gives an empirical cutoff of **28.5 Å** -- matching lorem-jax-sn2-lr to within 0.1 Å, as expected since it is a numerically-validated ($\sim 2\text{e-}5$ eV / $\sim 3\text{e-}5$ eV/Å) port of the exact same trained model, not a retraining. This supersedes the earlier note that no such loader existed (see notebook 15 's markdown for the full validation, including two non-PBC long-range bugs found and fixed along the way: a wrongly-inherited near-field exclusion radius, and a missing x2 training/production convention factor).

- Adding a dipole term to BPNN or PET does **not** measurably extend the cutoff (all still ≈ 5.5 Å); it changes the energetics (ΔE bonded $\rightarrow$ separated shifts by up to ~ 1.5 eV between variants) without adding genuine long-range force dependence -- i.e. the dipole head here is refining short-range energetics/dipole prediction, not introducing long-range coupling.
- All models remain **size-consistent** in the fragment_separation_test sense: interaction energy decays toward 0 at large separation (LOREM over its longer range, BPNN/PET over the shorter one) rather than diverging or plateauing at a nonzero value.
- Per-model details, including the reaction-coordinate energy/force curves and structure snapshots, are in notebooks 02_long_range_scan.ipynb , 03 - 11 , and 13 - 15 listed above.

In []: